

安徽大学《机器视觉视觉》实验报告（4）

学号： WA2224013 专业： 机器人工程 姓名及签字： 郭义月
实验日期： 2025.6.14 实验成绩： _____ 教师签字： _____

【实验名称】：碎纸片分割

【实验目的】：

1. 熟悉和掌握 matlab 基本编程环境
2. 熟悉和掌握基于 matlab 的计算机视觉编程基础
3. 通过 matlab 编程实现图像中碎纸片分割操作

【实验内容】

案例背景

图像拼接技术就是将数张有重叠部分的图像（可能是不同时间、不同视角或者不同传感器获得的）拼成一幅无缝的全景图或高分辨率图像的技术。相邻图像的配准及拼接是全景图生成技术的关键，有关图像配准技术的研究至今已有很长的历史，其主要的方法有以下两种：基于两幅图像的亮度差最小的方法和基于特征的方法。其中使用较多的是基于特征模板匹配特征点的拼接方法。该方法允许待拼接的图像有一定的倾斜和变形，克服了获取图像时轴心必须一致的问题，同时允许相邻图像之间有一定色差。

理论基础

全景图的拼接主要包括 5 个步骤：①图像获取；②图像的预拼接，即确定两幅相邻图像重合的较精确位置，为特征点的搜索奠定基础；③特征点的提取，即在基本重合位置确定后，找到待匹配的特征点；④图像矩阵变换及拼接，即根据匹配点建立图像的变换矩阵并实现图像的拼接；⑤图像的平滑处理。

【实验代码和结果】

实验代码 1：碎纸片拼接

1.1 拼接文件获取与合成

在图像拼接的过程中，为了将图片进行后续处理，需要抓取文件夹中的拼接图片后再进行后续处理。MATLAB 图像工具箱使用 `dir` 函数来获取文件列表，这个函数可以获得指定文件夹下的所有子文件夹和文件，并存放在一个文件结构的数组中。本案例以 `dir` 函数为例，其常见用法为

```
file_list = dir('.')
file_list = dir( '*.m' )
file_list = dir( '*.bmp' )
```

拼接过程中，为了把元胞数组中的多个矩阵合并成一个矩阵，需要将多个图片进行拼接处理，本案例中使用的函数为 `cell2mat`，其常见用法为

```
filename=cell2mat(image_list(ind));
```

在本实验中：

```
close all;
clear all;
clc;
% 获取文件列表
file_list= dir('../实验课数据/碎纸片拼接/*.bmp');
% file_list= dir('../实验课数据/校徽拼接/*.png');
if size(file_list,1)>0
    image_list = {file_list.name};
else
    error('no image')
end
```

1.2 拼接过程分析

1.文件获取

为了获取指定文件夹的文件列表，需要将多个碎纸片从指定文件夹中读取出来。利用 `dir` 函数能够获取指定文件夹的文件列表

file_list x						
19x1 struct 包含 6 个字段						
段	name	folder	date	bytes	isdir	datenum
	'000.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'001.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'002.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'003.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'004.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'005.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'006.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'007.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'008.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'009.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'010.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'011.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'012.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'013.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'014.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'015.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'016.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'017.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05
	'018.bmp'	'D:\Desktop\最近\dip_mv实验\实验课数据\碎纸片拼接'	'20-7月-2...	143638	0	7.3544e+05

2. 计算碎片特征

为了对每个单列矩阵进行特征值计算，需要将提取所有碎纸片图像矩阵的最左侧一列与最右侧一列的值并将其输出为一个 MATLAB 矩阵

代码为：

```
% 计算碎片特征
for ind=1:file_num
    filename=cell2mat(image_list(ind));
    filename = ['../实验课数据/碎纸片拼接/' filename];
%     filename = ['../实验课数据/校徽拼接/' filename];
    a=imread(filename);
    [row,col,len]=size(a);
    a_d=double(a);
    f =a_d(:,1)';
    tmp1=f.*f;
    tmp2=sqrt( sum(tmp1) );
    f =f/tmp2;
    one.left=f;
    one.left_std=std(f);
    f =a_d(:,end)';
    tmp1=f.*f;
    tmp2=sqrt( sum(tmp1) );
    f =f/tmp2;
    one.right=f;
    one.right_std=std(f);
    feature{ind}=one;
end
```

1.3 计算碎块的相似性矩阵

在进行步骤 2 之后，文件列表如图 14-48 所示，将计算出的每个碎片特征进行

对比分析，排除掉相同列矩阵后，输入到新建的两个矩阵中（分别以左侧为基准对比右侧矩阵，以右侧为基准对比左侧矩阵）。遇 见相同图片的左右两侧矩阵时，利用 `continue` 跳过乘积运算直接输出 0，避免出现错误。通过观察图 14-49 的相似性矩阵，读者可以得到，第 9 张碎纸片最左侧矩阵的为白色，即第 9 张碎纸片为最左侧碎纸片，通过观察图 14-50 的相似性矩阵，读者可以得到，第 7 张碎纸片最右侧矩阵为白色，即第 7 张碎纸片为最右侧碎纸片。

file_list		score_left																		
19x19 double		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
1	0	0.8737	0.8627	0.8891	0.8860	0.8753	0.9222	0.8835	0.9040	0.8803	0.8590	0.8995	0.9200	0.9006	0.9035	0.8690	0.8605	0.9813	0.8971	
2	0.8581	0	0.8772	0.8611	0.8784	0.8813	0.8997	0.8804	0.8860	0.8712	0.8702	0.8899	0.8810	0.8802	0.8793	0.8602	0.9765	0.8620	0.8741	
3	0.8772	0.8693	0	0.8990	0.8964	0.8921	0.9294	0.8858	0.8800	0.8838	0.9837	0.8977	0.9023	0.8878	0.9253	0.8728	0.8580	0.8716	0.8961	
4	0.8431	0.8662	0.8879	0	0.8786	0.8668	0.8913	0.8498	0.8706	0.8663	0.8647	0.8707	0.8812	0.8705	0.8671	0.9822	0.8553	0.8501	0.8699	
5	0.8954	0.9781	0.8893	0.8790	0	0.8798	0.9184	0.8933	0.8913	0.8724	0.8595	0.8878	0.9043	0.8776	0.9139	0.8661	0.8551	0.8624	0.8724	
6	0.8877	0.9072	0.9035	0.8915	0.9849	0	0.9572	0.8887	0.9277	0.8952	0.8918	0.9132	0.9294	0.9131	0.9236	0.8904	0.8914	0.8931	0.9158	
7	0.9827	0.8834	0.8673	0.8810	0.8754	0.8712	0	0.8867	0.8864	0.8642	0.8701	0.8831	0.8960	0.8944	0.9027	0.8410	0.8389	0.8566	0.8810	
8	0.8936	0.8816	0.9088	0.8953	0.8944	0.9161	0.9443	0	0.9032	0.8963	0.9035	0.9853	0.9251	0.9060	0.9208	0.8748	0.8861	0.8971	0.8992	
9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
10	0.8848	0.8889	0.8940	0.8812	0.8941	0.9832	0.9262	0.8827	0.8881	0	0.8957	0.9064	0.9221	0.8910	0.9009	0.8660	0.8752	0.8716	0.8931	
11	0.9008	0.8995	0.8928	0.9768	0.9011	0.8891	0.9296	0.8928	0.9017	0.8693	0	0.8941	0.9063	0.8903	0.9327	0.8726	0.8591	0.8769	0.8883	
12	0.8777	0.8756	0.8953	0.8839	0.9188	0.8839	0.9449	0.8804	0.9091	0.9063	0.8889	0	0.9327	0.9096	0.9099	0.8872	0.8735	0.8991	0.9831	
13	0.9118	0.9108	0.8962	0.9192	0.9058	0.9091	0.9553	0.9109	0.9044	0.8937	0.9085	0.9147	0	0.8947	0.9877	0.8794	0.8640	0.8935	0.8956	
14	0.8710	0.8756	0.8864	0.8795	0.8893	0.8896	0.9220	0.8866	0.8778	0.9817	0.8863	0.8967	0.9101	0	0.9063	0.8756	0.8660	0.8938	0.8948	
15	0.8944	0.8993	0.8808	0.8990	0.9280	0.8964	0.9514	0.8999	0.9803	0.8721	0.8841	0.9033	0.9274	0.9117	0	0.8742	0.8844	0.8998	0.9042	
16	0.9038	0.9053	0.9164	0.8971	0.9213	0.9146	0.9716	0.9099	0.9212	0.9054	0.8963	0.9275	0.9934	0.9299	0.9320	0	0.8793	0.9112	0.9316	
17	0.8725	0.8947	0.9908	0.8777	0.8928	0.8973	0.9233	0.8913	0.8851	0.8813	0.8733	0.9055	0.9124	0.8845	0.9006	0.8855	0	0.8611	0.8946	
18	0.8912	0.8768	0.8826	0.8927	0.8724	0.8835	0.9183	0.9753	0.8879	0.8787	0.8728	0.9003	0.9117	0.8897	0.9082	0.8494	0.8581	0	0.8738	
19	0.9094	0.8879	0.8799	0.8768	0.9107	0.8949	0.9395	0.8891	0.9159	0.8718	0.8821	0.9071	0.9207	0.9854	0.9030	0.8662	0.8765	0.8860	0	
20																				

19x19 double		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
1	0	0.8581	0.8772	0.8431	0.8954	0.8877	0.9827	0.8936	0.9212	0.8848	0.9008	0.8777	0.9118	0.8710	0.8944	0.9038	0.8725	0.8912	0.9094	
2	0.8737	0	0.8693	0.8662	0.9781	0.9072	0.8834	0.8816	0.9194	0.8889	0.8995	0.8756	0.9108	0.8756	0.8993	0.9053	0.8947	0.8768	0.8879	
3	0.8627	0.8772	0	0.8879	0.8893	0.9035	0.8673	0.9088	0.9221	0.8940	0.8928	0.8953	0.8962	0.8864	0.8808	0.9164	0.9908	0.8826	0.8799	
4	0.8891	0.8611	0.8990	0	0.8790	0.8915	0.8810	0.8953	0.9211	0.8812	0.9768	0.8839	0.9192	0.8795	0.8990	0.8971	0.8777	0.8927	0.8768	
5	0.8860	0.8784	0.8964	0.8786	0	0.9849	0.8754	0.8944	0.9466	0.8941	0.9011	0.9188	0.9058	0.8803	0.9280	0.9213	0.8928	0.8724	0.9107	
6	0.8753	0.8813	0.8921	0.8668	0.8798	0	0.8712	0.9161	0.9276	0.9832	0.8891	0.8839	0.9091	0.8896	0.8964	0.9146	0.8973	0.8835	0.8949	
7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
8	0.8835	0.8804	0.8858	0.8498	0.8933	0.8887	0.8867	0	0.9222	0.8827	0.8928	0.8804	0.9109	0.8866	0.8899	0.9099	0.8913	0.9753	0.8891	
9	0.9040	0.8860	0.8800	0.8706	0.8913	0.9277	0.8846	0.9032	0	0.8881	0.9017	0.9091	0.9044	0.8778	0.9803	0.9212	0.8851	0.8879	0.9159	
10	0.8803	0.8712	0.8838	0.8663	0.8724	0.8952	0.8642	0.8963	0.9147	0	0.8693	0.9063	0.8937	0.9817	0.8721	0.9054	0.8813	0.8787	0.8718	
11	0.8590	0.8702	0.9837	0.8647	0.8595	0.8918	0.8701	0.9035	0.9192	0.8957	0	0.8889	0.9085	0.8863	0.8841	0.8963	0.8733	0.8728	0.8821	
12	0.8995	0.8899	0.8977	0.8707	0.8878	0.9132	0.8831	0.9853	0.9449	0.9064	0.8941	0	0.9147	0.8967	0.9033	0.9275	0.9055	0.9003	0.9071	
13	0.9200	0.8810	0.9023	0.8812	0.9043	0.9294	0.8960	0.9251	0.9709	0.9221	0.9063	0.9327	0	0.9101	0.9274	0.9934	0.9124	0.9117	0.9207	
14	0.9006	0.8802	0.8878	0.8705	0.8776	0.9131	0.8944	0.9060	0.9415	0.8910	0.8903	0.9096	0.8947	0	0.9117	0.9299	0.8845	0.8897	0.9854	
15	0.9035	0.8793	0.9253	0.8671	0.9139	0.9236	0.9027	0.9208	0.9635	0.9009	0.9327	0.9099	0.9877	0.9063	0	0.9320	0.9006	0.9082	0.9030	
16	0.8690	0.8602	0.8728	0.9822	0.8661	0.8904	0.8410	0.8748	0.8987	0.8660	0.8726	0.8872	0.8794	0.8756	0.8742	0	0.8855	0.8494	0.8662	
17	0.8605	0.9765	0.8580	0.8553	0.8551	0.8914	0.8389	0.8861	0.8935	0.8752	0.8591	0.8735	0.8640	0.8660	0.8844	0.8793	0	0.8581	0.8765	
18	0.9813	0.8620	0.8716	0.8501	0.8624	0.8931	0.8566	0.8971	0.9152	0.8716	0.8769	0.8991	0.8935	0.8938	0.8998	0.9112	0.8611	0	0.8860	
19	0.8971	0.8741	0.8961	0.8699	0.8724	0.9158	0.8810	0.8992	0.9354	0.8931	0.8883	0.9831	0.8956	0.8948	0.9042	0.9316	0.8946	0.8738	0	
20																				

1.4 计算块间相似性

利用步骤 3 中的两个相似性矩阵的最大元素值将各个碎纸片图像进行顺序排列（相似性矩阵元素值大于 0.95 为准），并将该顺序存储到新建的 `order` 列表之中。分析每张碎纸片图像的最两侧列表时，以左侧和右侧新建两个相似性矩阵是为了保证每个碎纸片都能够找到对象。

代码为：

```
score=zeros(1,file_num);
feature=cell(1,file_num);
% 计算碎块的相似性矩阵
score_left=zeros(file_num,file_num);
score_right=zeros(file_num,file_num);
std_thr=0.001;
for y=1:file_num % compare
    for x=1:file_num
```

```

        if y==x
            continue;
        end
        score_left(y,x)=feature{y}.left*feature{x}.right';
        score_right(y,x)=feature{y}.right*feature{x}.left';
        if feature{y}.left_std<std_thr
            score_left(y,x)=0;
        end
        if feature{y}.right_std<std_thr
            score_right(y,x)=0;
        end
    end
end
end

```

计算块间的相似性:

```

file_num=size(image_list,2);
%% 计算块间相似性
order=[1];
order_left=2:file_num; % order%第一个保证前后都有对象
while 1
    order
    if isempty(order_left)
        break;
    end
    left=order(1);right=order(end);
    score=score_left(left,:);
    [c,i]=max(score);
    if c(1)>0.95 && any(order_left==i(1))
        order=[i order];
        order_left(order_left==i(1))=[];
    end
    score=score_right(right,:);
    [c,i]=max(score);
    if c(1)>0.95 && any(order_left==i(1))
        order=[order i];
        order_left(order_left==i(1))=[];
    end
end
end

```

1.5 拼接

利用步骤 4 得到的纸片顺序，将每个碎纸片矩阵进行叠加处理并储存在图 14-54 的 b 矩阵中。主要运用了 cell2mat 函数，其作用为把元胞数组中的多个矩阵合并成一个矩阵。最后利用 imshow 输出拼接完成图片。

实验代码为:

```

%% 拼接
b=[];
for ind=1:file_num % 拼接
    filename=cell2mat(image_list(order(ind)));
    filename = ['./实验课数据/碎纸片拼接/' filename];
%    filename = ['./实验课数据/校徽拼接/' filename];
    a=imread(filename);
    [row,col,len]=size(a);
    b=[b a];
end
subplot 121;
imshow(b)
title('拼接后的图像')

%% 原图像
order=(1:19);
b = [];
for ind=1:file_num % 拼接
    filename=cell2mat(image_list(order(ind)));
    filename = ['./实验课数据/碎纸片拼接/' filename];
%    filename = ['./实验课数据/校徽拼接/' filename];
    a=imread(filename);
    [row,col,len]=size(a);
    b=[b a];
end
subplot 122;
imshow(b)
title('碎图像')

```

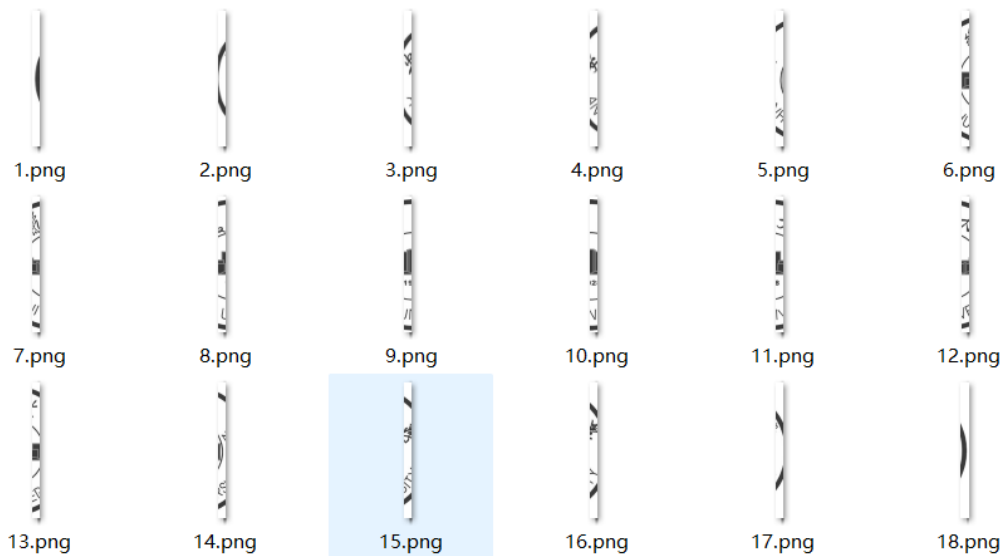
实验结果为:

以需要重新写 split 函数

代码为:

```
clc; clear; close all;
I = imread('../实验课数据/校徽拼接/Picture.png');
x = SplitArray(I, 18);
for i=1:18
    imwrite(x{i},['../实验课数据/校徽拼接/',num2str(i),'.png']);
end
function [x] = SplitArray(Array, n)
    row = size(Array, 1);
    column = size(Array, 2);
    x = cell(1, n);
    col_per_part = fix(column / n);
    for j = 1:n
        if j == n
            x{j} = Array(:, (j - 1) * col_per_part + 1:column);
        else
            x{j} = Array(:, (j - 1) * col_per_part + 1:j * col_per_part);
        end
    end
end
end
```

实验结果为:



2.2 校徽乱序

在拼接过程中, 由于 split 和命名是有顺序的, 如果不做乱序处理, 就会出现碎纸片拼接效果不明显的效果, 如下图所示:

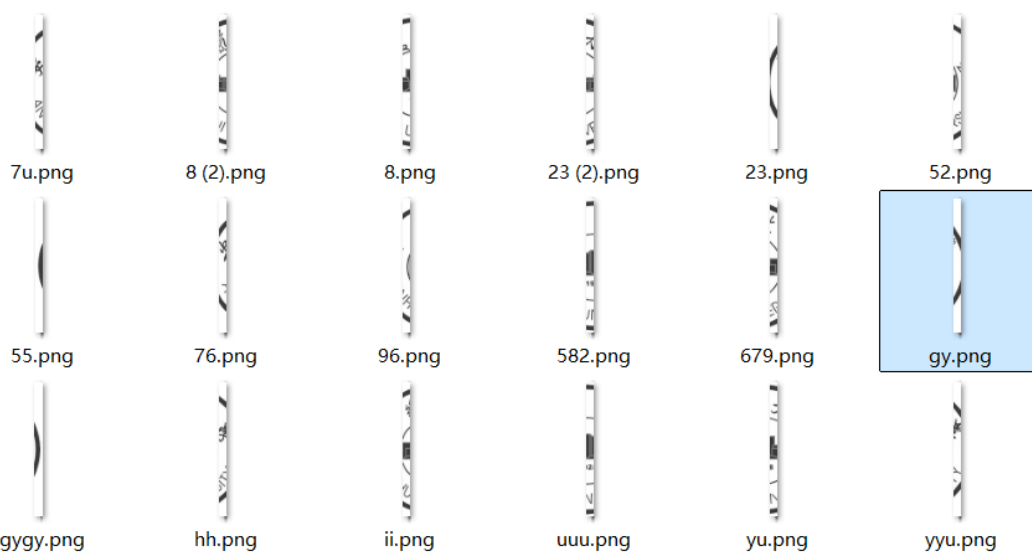
碎图像



拼接后的图像



碎图像和初始图像是类似的，乱序后，效果更加明显，可以用程序乱序，也可以手动乱序，因为只有 18 张图片，所以手动乱序一下~



乱序完成~

2.3 拼接

和实验 1 相比，需要修改路径，如下图所示，类似的路径需要修改 4 处

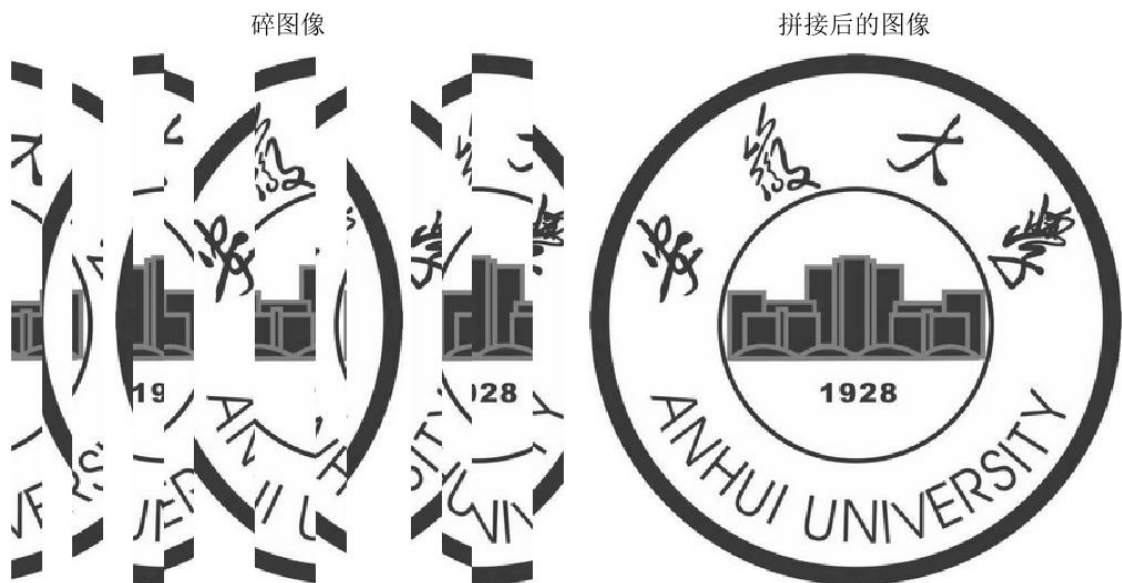
```

b=[];
for ind=1:file_num % 拼接
    filename=cell2mat(image_list(order(ind)));
    % filename = ['../实验课数据/碎纸片拼接/' filename];
    filename = ['../实验课数据/校徽拼接乱序/' filename];
    a=imread(filename);
    [row,col,len]=size(a);
    b=[b a];
end
imshow(b)

%% 原图像
order=(1:19);
b = [];
for ind=1:file_num % 拼接
    filename=cell2mat(image_list(order(ind)));
    % filename = ['../实验课数据/碎纸片拼接/' filename];
    filename = ['../实验课数据/校徽拼接乱序/' filename];
    a=imread(filename);

```

实验结果为：



乱序版校徽拼接完成~

【小结或讨论】

本次实验中，完成了碎纸片的处理

1. 学习了碎纸片拼接的五个步骤及 matlab 代码的编写

通过实验学习了图像拼接的基本步骤，包括图像获取、预拼接、特征点提取、图像矩阵变换及拼接、图像平滑处理，还通过 MATLAB 编程实现了这些步骤。

2.代码中路径的修改很重要

在实验过程中路径的正确设置对于代码的成功运行至关重要。路径错误可能导致文件无法正确读取或写入，影响实验结果，正确设置和修改路径是进行图像处理实验的基础。

3.发散到图像的拼接

实验 1 是文字的拼接，如果拼接图片会怎样。通过将碎纸片拼接的方法应用到图像拼接中，不仅完成了文字的拼接，还成功实现了校徽图像的拼接。

4.只能计算左右的相似度吗？如果是四面的拼图应该怎么算？

本次实验主要计算了碎纸片左右两侧的相似度。然而，对于四面的拼图需要考虑更多的方向和角度。可以思考一下